Wildcards: الأنواع العامة المرنة

في المحاضرة الماضية صنعنا `Box<T>` : صندوقاً واحداً يحمل أي نوع. اليوم نسأل سؤالاً يبدو بسيطاً وجوابه مفاجئ: **إذا كان الكلب حيواناً، فهل صندوق الكلاب هو صندوق حيوانات؟** من هذا السؤال نصل إلى الرمز `?` ومعاملات النوع بشرط. اقرأ بالترتيب.

ماذا سنتعلم؟ لماذا لا ترث الأنواع العامة (Invariance)، ثم `?` و `? extends` و `? super` ، ثم قاعدة PECS، ثم معاملات النوع بشرط `T extends` ومتى نفضلها على الـ wildcard.

المراجع: فصل Generics من كتاب Thinking In Java ودليل Oracle Wildcards Tutorial.

1 شغلتان مختلفتان: الوراثة والـ Generics

قبل أي شيء لازم نفصل بين فكرتين، لأن الخلط بينهما هو سبب الحيرة:

	الوراثة (Inheritance)	الـ Generics
العلاقة بين...	**حيوانين**	**صناديق**
مثال	`Dog` يرث من `Animal`	`Box<Dog>` و `Box<Animal>`
المعنى	الكلب **هو** حيوان	`Box<Dog>` صندوق «كلاب فقط»، و `Box<Animal>` صندوق «أي حيوان»

Wildcards.java

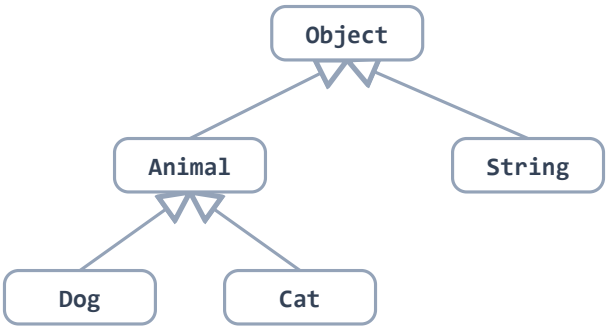
```
class Animal { String name; }
class Dog extends Animal { }
class Cat extends Animal { }

class Box<T> { T value; }

Animal a = new Dog();  // ok
```

الأب: له اسم
Dog يرث من Animal
Cat يرث من Animal
صندوق عام من الجلسة السابقة
كلب بمتغير حيوان: مقبول

بسبب الوراثة، متغير من نوع `Animal` يستطيع أن يحمل كائن `Dog` (السطر 7). أما `Box<T>` فهو صندوق، والحرف `T` مكان فاضي لنوع الشيء الذي بداخله، وأنا أختاره وقت الاستعمال.



الأسهم تعني «يرث من»: هذه علاقة بين الحيوانات فقط

السؤال: الكلب هو حيوان (وراثة). لكن هل **صندوق الكلاب** هو **صندوق حيوانات**؟ السؤال عن الصناديق، وليس عن الحيوانات.

2 مفاجأة: الصناديق لا ترث من بعضها

Wildcards.java

```
Box<Dog> bd = new Box<>();
Box<Animal> ba = bd;  // compile error
```

صندوق كلاب

خطأ ترجمة: Box<Dog> ليس Box<Animal>

الجواب المفاجئ: **لا**. علاقة «هو» موجودة بين الحيوانات فقط، ولا تنتقل إلى الصناديق التي تحملها. تخيّل قفصين: قفص لافتته «كلاب فقط» وقفص لافتته «أي حيوان». الكلب حيوان، لكن هذا لا يجعل القفصين نفس الشيء.

	الحيوانات (وراثة)	الصناديق (Generics)
المثال	`Dog` ⟶ `Animal`	`Box<Dog>` ⟶ `Box<Animal>`
النتيجة	الكلب هو حيوان	خطأ ترجمة: ليس نفسه

تسمى هذه الخاصية **Invariance** أي **الثبات**: كل نوع عام يبقى منفصلاً عن الآخر. للمقارنة: المصفوفات تقبل `Animal[] x = new Dog[3]` لكن الأنواع العامة لا تقبله، والسبب في الفقرة التالية.

3 لماذا يمنع المترجم ذلك؟

لنفترض أن Java **سمحت** بالسطر 2، فصار `ba` و `bd` يشيران إلى قفص واحد في الذاكرة. جرّب ماذا يحدث:

Wildcards.java

```
Box<Dog> bd = new Box<>();
Box<Animal> ba = bd;  // suppose allowed
ba.set(new Cat());     // fine for Animal box
Dog d = bd.get();      // gets a Cat!
```

نفترض أن المترجم سمح

مقبول: ba صندوق حيوانات

bd يتوقع Dog ويحصل على Cat!

السطر	ماذا يحدث؟
2	يصبح `ba` و `bd` يشيران إلى **الصندوق نفسه**
3	مقبول لأن `ba` صندوق حيوانات، فنضع قطة داخله
4	`bd` يعتقد أن الصندوق يحوي كلاباً، فيطلب كلباً ويحصل على قطة، وينهار البرنامج

لذلك يمنع المترجم السطر 2 من الأساس، حماية لأمان الأنواع (Type Safety)، وهذا ما تعلمناه في الجلسة السابقة.

❓ **فكّر: هل المصفوفات تعاني من المشكلة نفسها؟**

نعم، لكنها تكتشف الخطأ وقت التشغيل فقط. أما الأنواع العامة فتكتشفه وقت الترجمة، وهذا أفضل.

4 المشكلة العملية

نريد تابع تطبع اسم الحيوان الموجود في أي صندوق حيوانات. إذا كتبنا الوسيط `Box<Animal>` فلن تقبل إلا صندوق الحيوانات:

Wildcards.java

```
static void printName(Box<Animal> b) {
    System.out.println(b.get().name);
}
printName(new Box<Animal>());  // ok
printName(new Box<Dog>());     // error
printName(new Box<Cat>());     // error
```

الوسيط صندوق حيوانات فقط

مقبول

مرفوض (ثبات)

مرفوض (ثبات)

الحل البدائي أن نكتب تابع لكل نوع: واحدة للكلاب وواحدة للقطط... وهذا تكرار نريد الابتعاد عنه.

الفكرة: بدل أن نكتب نوعاً محدداً بين `<>`، نكتب علامة استفهام `?`. نقرؤها **«أي نوع، لا يهمني»**. هذه العلامة اسمها **Wildcard** أي البطاقة البيضاء: بطاقة فاضية تنطبق على أي نوع.

ما نكتبه	معناه
`Box<Animal>`	صندوق **حيوانات** فقط
`Box<?>`	صندوق من **أي نوع** (لا يهمني ما بداخله): `Box<Dog>` و `Box<Cat>` و `Box<String>` و `Box<Integer>`
`Box<? extends Animal>`	أي نوع **بشرط أن يكون حيواناً** (نشرحه بعد قليل)

5 الرمز ? غير المحدد

Wildcards.java

```
static void show(Box<?> b) {
    Object o = b.get();
    b.set("x");  // compile error
    b.set(null); // ok
}
show(new Box<String>());
show(new Box<Integer>());
```

صندوق لأي نوع

القراءة تعطي Object

الكتابة مرفوضة

null فقط مقبول

مقبول

مقبول

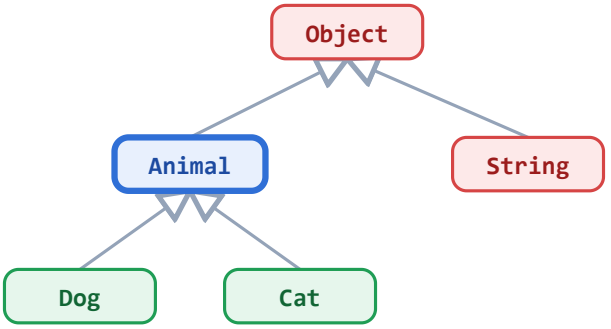
العملية	النتيجة	السبب
الاستدعاء بأي صندوق	مقبول	`Box<?>` صندوق لا نعرف نوع محتواه
القراءة `b.get()`	مقبولة	لكن النتيجة `Object` فقط
الكتابة `b.set("x")`	مرفوضة	المترجم لا يعرف هل الصندوق يقبل نصاً
الكتابة `b.set(null)`	مقبولة	`null` يقبل في أي نوع

6 الحد الأعلى: ? extends

Wildcards.java

```
static void printName(Box<? extends Animal> b) {
    Animal a = b.get();
    System.out.println(a.name);
}
printName(new Box<Dog>());     // ok
printName(new Box<Cat>());     // ok
printName(new Box<Animal>());  // ok
printName(new Box<String>());  // error
```

Animal أو أبناؤه
القراءة كـ Animal آمنة
مقبول
مقبول
مقبول
String ليس حيواناً



الأخضر: مقبول · الأحمر: مرفوض · الأزرق: السقف

الجزء	معناه
`?`	أي نوع (مجهول)
`extends Animal`	**الشرط**: بشرط أن يرث من `Animal` (وراثة)

إذن `Box<? extends Animal>` تعني صندوقاً محتواه `Animal` أو أي صف يرث منه. سميت **الحد الأعلى** لأن `Animal` هو السقف الأعلى للأنواع المقبولة. ما فوقه مثل `Object` وما لا علاقة له مثل `String` مرفوض. ونستطيع قراءة المحتوى كـ `Animal` لأنه مضمون أنه حيوان من نوع ما.

7 قاعدة extends: قراءة فقط

Wildcards.java

```
Box<? extends Animal> b = new Box<Dog>();
Animal a = b.get();        // ok
b.set(new Dog());          // error
b.set(new Animal());       // error
```

قراءة: مقبولة
قد يكون Box<Cat>
قد يكون Box<Dog>

المترجم **لا يتذكر** أن `b` صندوق كلاب؛ يعرف فقط «صندوق حيوانات من نوع ما». فقد يكون في الحقيقة أياً من هذه الثلاثة، فننظر ماذا يحدث لكل عملية:

النوع الحقيقي	`b.get()`	`b.set(new Dog())`	`b.set(new Animal())`
`Box<Dog>`	Animal ✓	✓	✗
`Box<Cat>`	Animal ✓	✗ كلب في صندوق قطط	✗
`Box<Animal>`	Animal ✓	✓	✓

القراءة نجحت مع **الثلاثة** فهي مقبولة. أما الكتابة فنجحت مع بعضها وفشلت مع بعضها، والمترجم لا يعرف أيها الحقيقي، فيرفض **كل كتابة** (إلا `null`).

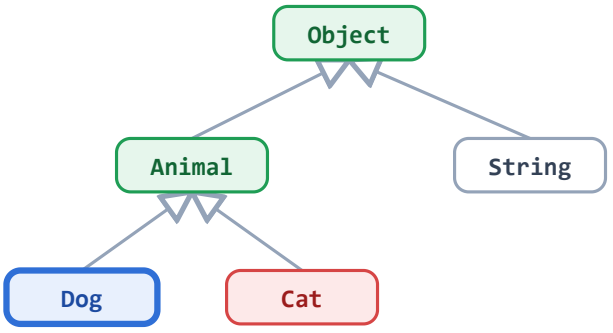
> مع `? extends` **نقرأ ولا نكتب**. نسمي هذا الصندوق **منتجاً Producer** لأنه يعطينا قيماً فقط.

8 الحد الأدنى: ? super

Wildcards.java

```
static void addDog(Box<? super Dog> b) {
    b.set(new Dog());
}
addDog(new Box<Dog>());      // ok
addDog(new Box<Animal>());   // ok
addDog(new Box<Object>());   // ok
addDog(new Box<Cat>());      // error
```

Dog أو أي صف أعلى منه
كتابة كلب آمنة
مقبول
مقبول
مقبول
Box<Cat> لا يقبل كلباً



نبدأ من Dog (القاع) ونصعد: Dog و Animal و Object

الجزء	معناه
`?`	أي نوع (مجهول)
`super Dog`	**الشرط**: بشرط أن يكون `Dog` أو أحد آبائه

النوع	`Box<? super Dog>` تقبله؟	السبب
`Box<Dog>`	✓ نعم	Dog نفسه
`Box<Animal>`	✓ نعم	Animal أب لـ Dog
`Box<Object>`	✓ نعم	Object أب لـ Animal
`Box<Cat>`	✗ لا	Cat ليس Dog ولا أباً له
`Box<String>`	✗ لا	لا علاقة له بـ Dog

سميت **الحد الأدنى** لأن `Dog` هو القاع: لا نقبل شيئاً تحته. والسبب أن كل صندوق مقبول يقبل بأمان وضع كلب داخله: الكلب حيوان وهو كائن.

9 قاعدة super: كتابة فقط

Wildcards.java

```
Box<? super Dog> b = new Box<Animal>();
b.set(new Dog());      // ok
Object o = b.get();    // ok
Dog d = b.get();       // error
```

كتابة: مقبولة
القراءة كـ Object فقط
قد يكون Box<Object>

المترجم لا يعرف النوع الحقيقي لـ `b` ؛ يعرف فقط «Dog أو أحد آبائه». فننظر ماذا يحدث لكل عملية مع الاحتمالات الثلاثة:

النوع الحقيقي	`b.set(new Dog())`	`b.get()` كـ Object	`b.get()` كـ Dog
`Box<Dog>`	✓	✓	✓
`Box<Animal>`	✓	✓	✗
`Box<Object>`	✓	✓	✗

الكتابة نجحت مع **الثلاثة** فهي مقبولة، والقراءة كـ Object مقبولة. أما القراءة كـ Dog فنجحت مع بعضها فقط، فتُرفض (السطر 4).

> مع `? super` **نكتب**. نسمي هذا الصندوق **مستهلكاً Consumer** لأنه يستقبل قيماً منا. الخلاصة: **extends للقراءة و super للكتابة**.

10 قاعدة PECS

الاختصار PECS هو: **P**roducer **E**xtends, **C**onsumer **S**uper.

الصندوق	دوره	نكتب
يعطيك قيماً (تقرأ منه)	Producer	`? extends`
يستقبل منك قيماً (تكتب فيه)	Consumer	`? super`
تقرأ وتكتب معاً	لا wildcard	نوع محدد مثل `Box<Animal>`

Wildcards.java

```java
static void copy(Box<? extends Animal> from,
                 Box<? super Animal> to) {
    to.set(from.get());
}
copy(new Box<Dog>(), new Box<Animal>());  // ok
copy(new Box<Dog>(), new Box<Object>());  // ok
```

المصدر منتج: extends
الهدف مستهلك: super
نقرأ ثم نكتب
مقبول
مقبول

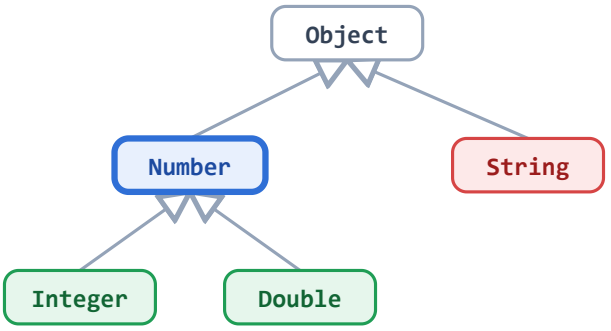
في التابع `copy` : المصدر `from` منتج فهو `extends` ، والهدف `to` مستهلك فهو `super` . نقرأ حيواناً من المصدر ونكتبه في الهدف، وهذا آمن.

11 Wildcard مع الأنواع الرقمية

Wildcards.java

```java
static double twice(Box<? extends Number> b) {
    return b.get().doubleValue() * 2;
}
twice(new Box<Integer>(5));
twice(new Box<Double>(2.5));
twice(new Box<String>("x"));  // error
```

أي صندوق عددي
doubleValue() من Number
مقبول
مقبول
String ليس عدداً



نفس الفكرة مع `Number` وأبنائه `Integer` و `Double` . نقرأ القيمة كـ `Number` فنستدعي `doubleValue()` . ومن غير الـ wildcard كنا سنحتاج تابع لكل نوع عددي.

12 معاملات النوع بشرط

نريد تابع عامة تطبع اسم أي حيوان. نكتبها عامة `<T>` لكن داخلها لا يعرف المترجم ما هو `T` ، فيعامله كأنه `Object` :

Wildcards.java

```java
static <T> void printName(T a) {
    System.out.println(a.name);  // error
}
```

T أي نوع
T يعامل كـ Object: لا name

المترجم محق: `T` قد يكون `String` أو أي نوع ليس له اسم. نحتاج أن نخبره أن `T` نوع من الحيوانات فقط.

الحل: نضيف `extends Animal` بعد `T` :

Wildcards.java

```java
static <T extends Animal> void printName(T a) {
    System.out.println(a.name);  // ok
}
printName(new Dog("Rex"));  // ok
printName(new Cat("Tom"));  // ok
printName("text");          // error
```

T حيوان أو ابن حيوان
name مضمون
مقبول
مقبول
String ليس حيواناً

`extends` **تعمل مع الصف ومع الواجهة (interface):** نكتب `T extends Animal` مع **الصف** (class)، و `T extends Pet` مع **الواجهة** (interface). في الحالتين نكتب `extends` ، ولا نكتب `implements` هنا.

13 T يحفظ النوع: ترجع نفس نوع المدخل

Wildcards.java

```java
// without T: returns Animal only
static Animal first(Animal a, Animal b) { return a; }
Dog d = first(new Dog("A"), new Dog("B"));  // error

// with T: returns the same type it receives
static <T extends Animal> T first(T a, T b) { return a; }
Dog d = first(new Dog("A"), new Dog("B"));  // ok
```

ترجع Animal دائماً
Animal لا يتحول إلى Dog
ترجع نفس نوع المدخل
النتيجة Dog بدون Casting

هذا **تابع** (method) اسمه `first` . بدون `T` نوع الإرجاع `Animal` دائماً، فمع كلبين ترجع `Animal` ولا نستطيع حفظها في `Dog` بدون Casting. مع `T` ترجع `Dog` مباشرة.

ميزة `T` أن له **اسماً**، فنستعمله مرتين: في المدخل وفي الإرجاع. الـ wildcard بلا اسم، فلا تستطيع هذا.

14 شرطان معاً: T extends Animal & Pet

Wildcards.java

```java
interface Pet { void play(); }
class Dog extends Animal implements Pet { /* ... */ }
static <T extends Animal & Pet> void care(T x) {
    System.out.println(x.name);  // from Animal
    x.play();                    // from Pet
}
care(new Dog("Rex"));  // ok: Animal and Pet
care(new Cat("Tom"));  // error: Cat is not a Pet
```

شرطان معاً: صف ثم واجهة
من Animal
من Pet
Dog يحقق الشرطين
Cat ليس Pet

النوع	Animal؟	Pet؟	يقبله التابع care؟
`Dog`	✓	✓	✓ مقبول
`Cat`	✓	✗	✗ مرفوض

يمكن وضع أكثر من شرط بالرمز `&` على أن يأتي **الصف أولاً ثم الواجهات**، ويجب أن يحقق النوع كل الشروط معاً. لذلك داخل التابع نستعمل `x.name` (من Animal) و `x.play()` (من Pet).

15 Wildcard أم معامل نوع T؟

Wildcards.java

```java
// wildcard: types may differ
static void show(Box<?> b) { }
show(new Box<String>("a"));   // ok
show(new Box<Integer>(5));    // ok

// T: same type in both places
static <T> void swapValues(Box<T> a, Box<T> b) {
    T tmp = a.get();
    a.set(b.get());
    b.set(tmp);
}
swapValues(new Box<String>("a"), new Box<String>("b"));  // ok
swapValues(new Box<String>("a"), new Box<Integer>(5));   // error
```

النوع يمكن أن يختلف
مقبول
مقبول
النوع نفسه في المكانين
T يحمل نوع الصندوق
String مع String
String مع Integer: مرفوض

	Wildcard `?`	معامل `T`
متى؟	لا أحتاج أن أسمي النوع	يتكرر النوع نفسه، أو يطابق الإرجاعُ المدخلَ
مثال	`show(Box<?> b)`	`swapValues(Box<T>, Box<T>)`
`String` مع `Integer`	مقبول	**مرفوض**: التبديل يضع نصاً داخل صندوق أعداد

16 المفاضلة: مرونة التابع أم وضوحه

Wildcards.java

```java
// flexible, longer signature
static void copy(Box<? extends Animal> from, Box<? super Animal> to) { }
// clearer, less flexible
static <T extends Animal> void copy2(Box<T> from, Box<T> to) { }

copy(dogBox, animalBox);    // ok
copy2(dogBox, animalBox);   // error
copy2(dogBox, dogBox2);     // ok
```

مرن: توقيع أطول
أوضح: نوع واحد للاثنين
Dog إلى Animal: مقبول
T لا يكون Dog وAnimal معاً
النوع نفسه: مقبول

عندنا **تابعان** للمهمة نفسها: `copy` بالـ wildcard و `copy2` بمعامل النوع. ولنفرض `dogBox` من نوع `Box<Dog>` و `animalBox` من نوع `Box<Animal>`.

الاستدعاء	النتيجة
`copy(dogBox, animalBox)`	**مقبول**: المصدر extends والهدف super
`copy2(dogBox, animalBox)`	**مرفوض**: `T` لا يكون Dog وAnimal معاً
`copy2(dogBox, dogBox2)`	**مقبول**: النوع نفسه

قاعدة عملية: wildcard للوسائط عندما لا أحتاج تسمية النوع، و `T` عندما يرتبط وسيطان أو وسيط وقيمة الإرجاع، و**لا wildcard في نوع الإرجاع**.

17 ملاحظات وقيود

Wildcards.java

```java
Box<?> a = new Box<String>();  // ok
Box<?> b = new Box<?>();       // error

// invalid:
// Box<? extends A super B> c;
```

مقبول

ممنوع new مع ?

ممنوع جمع extends وsuper

- الـ wildcard يستعمل في نوع المتغير أو الوسيط فقط، ولا يجوز `new Box<?>()` .
- لا يجوز جمع `extends` و `super` في wildcard واحد.
- `Box` تعادل `Box<?>`، والرمز `?` وحده لا يعني الصف Object بل نوعاً مجهولاً.
- في الجلسات القادمة سنستعمل wildcards كثيراً مع المجموعات Collections.

18 التمارين المحلولة

نحل الستة بالترتيب. الصفوف `Animal` و `Dog` و `Cat` و `Box<T>` معرفة في التمرين 1 ونستعملها في الباقي.

التمرين 1: إثبات أن `Box<Dog>` ليس `Box<Animal>`

تعريف الصفوف التي سنعمل عليها، وإثبات أن `Box<Dog>` ليس `Box<Animal>`.

Ex1.java

```
class Animal {
    String name;
    Animal(String name) { this.name = name; }
}
class Dog extends Animal {
    Dog(String name) { super(name); }
}
class Cat extends Animal {
    Cat(String name) { super(name); }
}
class Box<T> {
    private T value;
    Box(T value) { this.value = value; }
    T get() { return value; }
    void set(T value) { this.value = value; }
}
public class Ex1 {
    public static void main(String[] args) {
        Animal a = new Dog("Rex");
        Box<Dog> bd = new Box<>(new Dog("Rex"));
        // Box<Animal> ba = bd;  // compile error
        System.out.println(a.name);
        System.out.println(bd.get().name);
    }
}
```

وراثة عادية
صندوق كلاب
Invariance

السطر	الشرح
1 - 4	الصف الأب `Animal` فيه اسم ومنشئ
5 - 10	`Dog` و `Cat` يرثان من `Animal` ويمرران الاسم للأب
11 - 16	الصف العام `Box<T>` من الجلسة السابقة
19	وراثة عادية: كلب داخل متغير حيوان، مقبول
20	صندوق كلاب `Box<Dog>`
21	مرفوض: رغم أن `Dog` هو `Animal`، فإن `Box<Dog>` ليس `Box<Animal>` (Invariance)
22، 23	نطبع الاسم من المتغيرين

المخرج:

```
Rex
Rex
```

التمرين 2: تابع تقبل أي صندوق بـ ?

تابع تقبل أي صندوق وتعرض محتواه.

Ex2.java

```java
public class Ex2 {
    static void show(Box<?> b) {
        Object o = b.get();
        // b.set("x");  // compile error
        System.out.println("Box contains: " + o);
    }
    public static void main(String[] args) {
        Box<String> s = new Box<>("Java");
        Box<Integer> n = new Box<>(42);
        show(s);
        show(n);
    }
}
```

تقبل أي صندوق
النتيجة Object
الكتابة مرفوضة

السطر	الشرح
2	`Box<?>` تقبل صندوقاً من أي نوع
3	القراءة ممكنة لكن النتيجة `Object` لأن النوع مجهول
4	الكتابة مرفوضة لأن المترجم لا يعرف نوع الصندوق
5	الطباعة تستدعي `toString` للقيمة
8، 9	صندوق نصوص وصندوق أعداد
10، 11	التابع نفسها قبلت النوعين

المخرج:

```
Box contains: Java
Box contains: 42
```

التمرين 3: القراءة بـ ? extends

قراءة من أي صندوق حيوانات.

Ex3.java

```
public class Ex3 {
    static void printName(Box<? extends Animal> b) {
        Animal a = b.get();
        System.out.println(a.name);
    }
    public static void main(String[] args) {
        Box<Dog> d = new Box<>(new Dog("Rex"));
        Box<Cat> c = new Box<>(new Cat("Tom"));
        Box<Animal> g = new Box<>(new Animal("Generic"));
        printName(d);
        printName(c);
        printName(g);
        // printName(new Box<String>("x"));  // error
    }
}
```

Animal أو أبناؤه

قراءة آمنة

String مرفوض

السطر	الشرح
2	الوسيط صندوق محتواه `Animal` أو أبناؤه
3	نقرأ القيمة كـ `Animal` وهذا آمن لأن كل الأنواع المقبولة حيوانات
4	نطبع الاسم
7 - 9	ثلاثة صناديق بأنواع مختلفة
10 - 12	التابع قبلت الأنواع الثلاثة بدل كتابة ثلاث توابع
13	`String` ليس حيواناً فيرفضه المترجم

المخرج:

```
Rex
Tom
Generic
```

التمرين 4: الكتابة بـ ? super

كتابة كلب في صندوق باستخدام `? super`.

Ex4.java

```
public class Ex4 {
    static void addDog(Box<? super Dog> b) {
        b.set(new Dog("Max"));
    }
    public static void main(String[] args) {
        Box<Dog> bd = new Box<>(null);
        Box<Animal> ba = new Box<>(null);
        Box<Object> bo = new Box<>(null);
        addDog(bd);
        addDog(ba);
        addDog(bo);
        System.out.println(bd.get().name);
        System.out.println(ba.get().name);
        // addDog(new Box<Cat>(null));  // error
    }
}
```

Dog أو أعلى

كتابة آمنة

Box<Cat> مرفوض

السطر	الشرح
2	الوسيط صندوق محتواه `Dog` أو أي صف أعلى منه
3	نكتب كلباً بأمان لأن كل الأنواع المقبولة تقبل كلباً
6 - 8	ثلاثة صناديق فارغة: كلب وحيوان وكائن
9 - 11	التابع قبلت الصناديق الثلاثة
12، 13	نقرأ من الصندوقين المحددين بنوعهما لنطبع الاسم
14	`Box<Cat>` لا يقبل كلباً فيرفضه المترجم

المخرج:

```
Max
Max
```

التمرين 5: النسخ بقاعدة PECS

نسخ قيمة بين صندوقين.

Ex5.java

```java
public class Ex5 {
    static void copy(Box<? extends Animal> from,
                     Box<? super Animal> to) {
        to.set(from.get());
    }
    public static void main(String[] args) {
        Box<Dog> source = new Box<>(new Dog("Rex"));
        Box<Animal> t1 = new Box<>(null);
        Box<Object> t2 = new Box<>(null);
        copy(source, t1);
        copy(source, t2);
        System.out.println(t1.get().name);
        // copy(t1, source);  // error
    }
}
```

المصدر منتج

الهدف مستهلك

نقرأ ونكتب

العكس مرفوض

السطر	الشرح
2	المصدر منتج نقرأ منه، لذلك `extends`
3	الهدف مستهلك نكتب فيه، لذلك `super`
4	نقرأ حيواناً من المصدر ونكتبه في الهدف، آمن في الحالتين
7	المصدر صندوق كلاب
8، 9	هدفان: صندوق حيوانات وصندوق كائنات
10، 11	النسخ مقبول لكليهما
13	العكس مرفوض: `Box<Dog>` لا يقبل أن نكتب فيه `Animal`

المخرج:

```
Rex
```

التمرين 6: معامل نوع بشرط `T extends Animal`

تابع عامة بمعامل نوع بشرط.

Ex6.java

```java
public class Ex6 {
    static <T extends Animal> void printName(T a) {
        System.out.println(a.name);
    }
    static <T extends Animal> T first(T a, T b) {
        return a;
    }
    public static void main(String[] args) {
        Dog d1 = new Dog("Rex");
        Dog d2 = new Dog("Max");
        printName(d1);
        Dog d = first(d1, d2);
        System.out.println(d.name);
        // printName("text");  // error
    }
}
```

T حيوان أو ابنه

name مضمون

ترجع T نفسه

النتيجة Dog مباشرة

String مرفوض

السطر	الشرح
2	`T extends Animal` تعني أن `T` حيوان أو من أبنائه
3	نصل إلى `name` لأن الشرط يضمن وجوده
5	`first` تأخذ وسيطين من النوع `T` نفسه وترجع `T`
6	نرجع الوسيط الأول
11	استدعاء مقبول لأن `Dog` حيوان
12	النتيجة `Dog` مباشرة بدون Casting لأن `T` أصبحت `Dog`
14	`String` ليس حيواناً فيرفضه المترجم

المخرج:

```
Rex
Rex
```

❓ فكّر: لماذا لا نستعمل wildcard في نوع الإرجاع؟

لأن من يستدعي التابع لن يعرف النوع الدقيق للنتيجة، وسيضطر إلى Casting. معامل النوع `T` يحل ذلك.

❓ فكّر: متى لا أستعمل wildcard أبداً؟

عندما أقرأ وأكتب معاً في الصندوق نفسه. هنا أحتاج نوعاً محدداً أو معامل نوع.

19 الخلاصة

أريد...	أكتب	مثال
أن أقبل أي صندوق	`?`	`Box<?>`
أن أقرأ من صندوق (منتج)	`? extends`	`Box<? extends Animal>`
أن أكتب في صندوق (مستهلك)	`? super`	`Box<? super Dog>`
أن يتكرر النوع أو يرتبط بالإرجاع	`<T extends ...>`	`<T extends Animal> T first(T a, T b)`

الفكرة في جملة: الأنواع العامة لا ترث (Invariance)، فنستعمل `?` للمرونة: `extends` للقراءة و `super` للكتابة (PECS)، و `T extends` عندما أحتاج اسماً للنوع.